FIELD GUIDE / 01

让两台电脑接上同一份工作

GitHub 仓库界面、提交原理与 PHM 双机协作

开工先拉取
收工先提交，再推送

A 电脑 → GitHub / PHM → B 电脑
每台电脑独立克隆，通过同一分支交接。

按你现在的问题阅读

看懂页面	第 2-4 页：标签、按钮、文件和历史入口。
理解提交	第 5-6 页：工作区、暂存区、提交图与分支引用。
两台电脑接力	第 7-9 页：首次准备、日常命令、Desktop 操作。
出错后怎么办	第 10-16 页：分叉、冲突、stash、分支、恢复与环境。
随手查阅	第 17 页交接卡；第 18-19 页官方来源。

适用：个人研究代码仓库。示例使用 main 与 YOUR-USER/PHM，实际名称请自行核对。界面为重绘示意，不是已登录账号的真实截图。

INTERFACE / 02

先认出这九个区域

PHM 仓库首页示意；布局压缩显示，侧栏内容在此放到图底部。

依据：S01 / S02 | 完整链接见文末

① YOUR-USER / PHM　Private　Watch　Star　Fork ②

③ Code　Issues　Pull requests　Actions　Projects　Security　Insights　Settings

④ main ▼　branches　tags　Go to file　Add file　⑤ Code ▼

⑥ 最新提交说明　a1b2c3d　History / commits

⑦ src/　修复数据读取

tests/　增加样例检查

README.md　更新使用说明

⑧ README：安装、运行、数据说明

⑨ About / Releases / Contributors / Languages

① 仓库身份	owner、PHM、可见性：先确认账号、仓库和是否公开。
② 关注工具	Watch 管通知；Star 收藏；Fork 另建派生仓库。
③ 功能标签	Code 看文件；其他标签按任务使用，部分可能隐藏。
④ 当前分支	决定网页显示哪条分支；tags 是标记版本的入口。
⑤ 绿色 Code	复制 clone 地址、工具入口、Download ZIP 等。
⑥ 提交入口	查看提交说明、短编号与历史；入口随页面变化。
⑦ 文件列表	进入目录和文件；每行可显示影响该路径的最近提交。
⑧ / ⑨ 文档与侧栏	README 讲怎么用；About、Releases 等讲项目与版本。

INTERFACE / 03

顶部标签：什么时候点哪个

先用 Code、Issues、Pull requests，其他随项目需要启用。

依据：S01 / S09 / S12 / S13 | 完整链接见文末

Code	项目文件、分支、README、历史。日常查看 PHM 源码从这里开始。
Issues	记录缺陷、需求与任务。例如“CSV 读取时出现空值”。
Pull requests	比较分支、说明改动、看检查结果、请求合并。PR 和 git pull 是不同的操作。
Actions	已配置工作流的运行结果与日志。红叉可能是测试失败，不等于 push 失败。
Projects	看板或表格管理任务，例如待做、进行中、已完成。
Wiki / Discussions	可选的长文知识库与主题讨论区，未开启就可能不显示。
Security	安全策略、依赖或秘密相关功能；具体能力取决于权限、仓库与套餐。
Insights	贡献、提交和活动统计。统计量不能证明代码或实验质量。
Settings	配置、访问权限、规则等。只有拥有相应权限的人能看到对应选项。

> **个人仓库也可以用 PR**
>
> 自己提交、自己检查、自己合并，也能留下清楚的改动理由。是否需要审批、是否能直接 push，由仓库规则决定。

INTERFACE / 04

文件页与右侧栏怎么用

Code 标签页与绿色 Code 按钮同名，但作用不同。

依据：S02 / S11 / S16 | 完整链接见文末

分支下拉	A 推到了 feature 分支，网页选 main 就看不到那份改动。先核对分支。
Go to file / 搜索	按文件名或代码搜索定位；进入目录后面包屑可返回上层。
Add file	网页创建或上传文件，并在 GitHub 形成提交；本机之后也需要 pull。
绿色 Code 按钮	HTTPS / SSH 地址用于 clone。ZIP 只有该版本的源码快照，没有完整 .git 历史。
文件内容与历史	History 看该文件的修改记录；Blame 定位代码行关联的提交。Raw 看原始内容。
提交短编号	点击进入改动明细；用它核对“这次修改是否已到网页与 B 电脑”。
README / About	README 说明安装运行；About 放项目摘要、主题和相关链接。
Releases / Tags	Tag 指向版本提交；Release 额外提供版本说明和可下载资产，二者不相同。
Packages / 其他侧栏	软件包、贡献者、语言、部署等按内容和设置显示。语言比例不代表工作量。

Private 代表受访问控制，仍可能有协作者或获授权应用。源码归档也不保证包含外部数据和所有 LFS 实体。

MENTAL MODEL / 05

提交不是上传

先理解一次修改在哪个位置，再记命令。

依据：S05 / S06 / S20 | 完整链接见文末

工作区
编辑器里保存后的文件 →

暂存区
git add 选入本次提交 →

本地仓库
git commit 形成版本 →

GitHub
git push 上传并更新分支

add 之后再改文件，会发生什么?

例如 README 原来写“采样 100”。你改成“采样 200”并执行 add，再改成“采样 300”。此时 commit 默认记录的是已暂存的“采样 200”。要把 300 纳入提交，需要再次 add。

```
git diff                # 看未暂存差异
git add -- README.md    # 暂存这个文件
git diff --cached       # 看本次实际要提交的内容
git commit -m "更新说明" # 只在本机形成提交
git push                # 上传已有提交
```

提交对象记录什么

根目录 tree、父提交、作者与提交者信息、提交说明等。tree 关联文件与目录对象。Git 逻辑上记录项目快照，并复用相同内容；底层还会压缩存储。

提交编号来自对象内容，不是文件名或时间序号。首次提交没有父提交，普通提交通常一个父提交，合并提交可有多个父提交。push 不会自动保存编辑器文件、自动 commit、测试或部署。

main、origin/main 与 HEAD

三个位置需要分清：本地分支、远程跟踪引用、服务器上的分支。

依据：S04 / S05 / S06 | 完整链接见文末

本地 main

本机分支，指向一个提交

→

origin/main

最近 fetch 掌握的远程位置

→

GitHub main

服务器实际分支，可被 A 或 B 更新

origin	通常是远程仓库别名；不是分支，也不是 GitHub 固定保留字。
HEAD	通常指向当前分支；切换分支会改变当前检出的代码。
fetch	获取对象与远程引用信息，通常不改变当前工作区。
pull	先 fetch，再整合到当前分支；不是单纯下载文件。
--ff-only	只允许快进更新；分叉时停止，让你先判断如何整合。
push	把本地提交发给远程，并请求更新相应分支引用。

```
C0 --- C1 --- C2
      main     GitHub main
git pull --ff-only
C0 --- C1 --- C2
              main / HEAD
```

上图表示快进：本地没有独有提交，只需把 main 前移。origin/main 离线时可能过期；先 fetch 再用它判断进度。

SETUP / 07

两台电脑第一次怎么准备

同一账号、同一仓库、两份独立克隆；路径可以不同。

依据：S10 / S17 / S19 | 完整链接见文末

选一条路线，保持简单

想少用命令	GitHub Desktop 登录账号 → Clone repository → 选本地目录。
愿意学命令	HTTPS + Git Credential Manager，在浏览器认证；不必先手工创建 PAT。
已有 SSH 习惯	每台电脑生成自己的密钥，只把公钥加到账号，不复制私钥。

```
git --version
git config --global user.name "你的提交显示名"
git config --global user.email "你的提交邮箱"
# 在已有父目录运行；替换 YOUR-USER
git clone https://github.com/YOUR-USER/PHM.git PHM
Set-Location -LiteralPath .\PHM
git remote -v
git branch -vv
```

新电脑 clone，已有克隆 pull

clone 会建立本地仓库、origin 和默认检出分支的跟踪关系。已经 clone 的目录不必再 init。已有同名文件夹时，先保留它，克隆到新目录再比较复制源码，不覆盖 .git。

作者姓名与邮箱不等于登录权限。可用 GitHub 实际提供的 noreply 邮箱。HTTPS、SSH 只是认证与传输选择，不改变 add、commit、pull、push 的核心含义。

A → GitHub → B：每天照着做

本页前提：当前是目标分支、工作区干净、已配置上游；每段报错就停。

依据：S04 / S20 | 完整链接见文末

01 A 开工：确认位置，拉取后再编辑

```
git rev-parse --show-toplevel
git branch --show-current
git status
git pull --ff-only
```

02 A 收工：保存、检查、提交、推送

```
git status
git diff
git add -- "实际要提交的文件路径"
git diff --cached
git commit -m "修复传感器数据读取"
git push
git log -1 --oneline
```

03 B 接手：同一分支拉取，核对提交编号

```
git branch --show-current
git status
git pull --ff-only
git log -1 --oneline
```

网页选中相同分支确认 A 的提交已出现；B 核对短编号后再编辑。B 收工同样 commit + push，返回 A 同样先 pull。新分支首次 push 需要 -u origin 分支名。

> **clean 不等于已上传**
>
> working tree clean 只表示没有未提交修改；本地仍可能领先远程。必须确认 push 成功，以及网页同分支出现这次提交。

不用命令：Desktop 双机操作

把 A、B 当作两个独立客户端；两边都要同步。

依据：S03 / S20 | 完整链接见文末

首次 B 接入	File → Clone repository，选择 PHM，选保存目录，Clone。
开工看哪一份	Current Repository 选 PHM；Current Branch 选要接手的分支。
获取更新	先 Fetch origin；有远程更新时再 Pull origin。Fetch 本身不会把代码整合到工作区。
查看这次修改	保存代码后返回 Desktop，在 Changes 中看差异，勾选需要提交的文件。
形成本地版本	填写 Summary，点击 Commit to 当前分支。此时还没有上传。
上传交接	点击 Push origin，确认成功；必要时从网页核对分支与提交。
处理分叉	如果提示拉取、冲突或 rebase，读清提示，按正在进行的操作完成；不要盲点重试。
回另一台电脑	选择相同仓库与分支，再 Fetch 和 Pull，确认新提交出现在 History。

> **按钮随状态改变**
>
> Fetch origin、Pull origin、Push origin 会随状态显示。受保护分支可能要求走 PR。不同版本按钮位置会变，按标签和提示识别即可。

命令行与 Desktop 操作的是同一份克隆时，看到的是同一套 Git 历史。不要在两个不同的 PHM 文件夹来回操作，却以为它们会自行同步。

DIVERGENCE / 10

两台电脑都 commit 了怎么办

push 拒绝通常是在保护远程独有提交；先判断，再合并。

依据：S04 / S07 | 完整链接见文末

```
git status              # 先处理未提交修改
git branch backup/b-before-sync
# 备份名要唯一；它只保护当前已提交历史
git fetch origin
git rev-list --left-right --count "HEAD...@{upstream}"
```

0 / 0	同步；仍需另外查看是否存在未提交文件。
0 / 正数	本地落后，用 pull --ff-only。
正数 / 0	本地领先，检查后 push。
正数 / 正数	分叉，不能快进，下面二选一。

默认：merge 保留双方提交

```
git merge origin/main
# 若无冲突，检查后 git push
# 有冲突见下一页；合并成功后不再机械执行 rebase
```

前提：当前确实在 main、刚完成 fetch，工作区干净且没有其他进行中的操作。已经共享的提交优先保持历史稳定；备份分支不包含未提交文件。

> **可选：未发布的本地提交才考虑 rebase**
>
> 若只是本地尚未推送的普通提交，可另选 git rebase origin/main，让它接在远程提交之后。会产生新编号；不要随意 rebase 已共享历史。正常完成后再 push，无需强推。

冲突不是二选一按钮题

示例数值只用于教学，真实参数要按项目需求判断。

依据：S04 / S07 / S08 | 完整链接见文末

```
<<<<<<< HEAD
sample_rate = 200
=======
sample_rate = 100
>>>>>>> origin/main
```

Git 只能发现部分文本冲突，不知道采样率应该多少。先读上下文与双方改动目的，保留正确实现并删掉冲突标记。rebase 时 Current / Incoming 的含义容易误判，应以实际内容为准。

```
git status
git diff --check
git add -- "实际冲突文件路径"
git diff --cached
```

merge 冲突	所有冲突解决并暂存后 git commit；检查后 push。放弃本次操作：git merge --abort。
rebase 冲突	git rebase --continue，可能重复多轮；全部完成后检查并 push。放弃：git rebase --abort。
stash apply 冲突	解决、暂存后作为普通修改 commit。没有 stash --continue，也不运行 rebase --continue。

> **自动合并成功也要检查**
>
> 双方改不同代码段，仍可能产生逻辑不兼容。按项目说明检查和测试。PDF、图片、模型等二进制文件需保留两份后人工选择或重新生成。

没 commit，想先拉取怎么办

先保存当前工作；stash 是本地抽屉，功能分支才适合跨电脑交接。

依据：S08 / S09 | 完整链接见文末

临时挪开修改：stash

```
git status
git stash push -u -m "同步前暂存"
git stash list
git pull --ff-only
# pull 成功后，确认要恢复的记录
git stash show --stat "stash@{0}"
git stash apply "stash@{0}"
git status
```

-u 包含未跟踪文件，不包括被忽略文件。apply 保留 stash 记录，恢复时可能冲突。pull 失败就先停止，处理分叉等问题后再恢复；不要反复 apply。

> **恢复完再考虑清理**
>
> 解决冲突、检查并提交或完成其他备份后，才可按需 git stash drop 对应记录。stash 不随普通 push 上传，B 无法从 GitHub 拉到 A 的 stash。

要带到 B 的未完成工作：功能分支

在适当功能分支保存阶段进度并 commit + push，B fetch 后跟踪同名分支。WIP 可以标记未完成状态，但仍要确认不含秘密与不应上传的数据。

若 A 离线，先在本地 commit，联网再 push；若 A 无法访问且没有备份，GitHub 无法凭空恢复未上传内容。

同一个功能，换电脑接着做

不必按电脑名分支；按功能分支，同一任务在 A、B 轮流推进。

依据：S09 | 完整链接见文末

A：从更新后的 main 创建功能分支

```
git switch main
git pull --ff-only
git switch -c feature/phm-reader
# 编辑、暂存、检查后
git commit -m "WIP: 数据读取第一阶段"
git push -u origin feature/phm-reader
```

B：第一次接手这个远程分支

```
git fetch origin
git branch -a
git switch --track origin/feature/phm-reader
# 仅当 B 尚无同名本地分支时使用
```

若 B 已有该本地分支：先 switch 到它，再 pull --ff-only。完成修改后正常 add、commit、push。回 A 也先在这个分支 pull。切换前先处理未提交修改。

> **功能完成：用 PR 合并到 main**
>
> Pull requests → New pull request。base 选 main，compare 选 feature/phm-reader；看差异与检查后按规则合并。两台电脑各自切回 main 并 pull。

PR 不会自动下载到本机。若以 squash 或 rebase 方式合并，旧分支历史可能不同；下一项任务从最新 main 新建分支，不在旧功能分支继续混用。

“更新不见了”按这个顺序查

越先确认仓库和分支，越少走弯路。

依据：S03 / S10 / S17 | 完整链接见文末

A 只保存 / 只 commit	在 A 看 status 和 log；完成 push，网页确认后 B 再 pull。
B 只 fetch	fetch 只获取远程信息；工作区干净时再 pull --ff-only。
网页与本地分支不同	branch --show-current、branch -vv 核对；网页下拉选同一分支。
编辑器打开错目录	rev-parse --show-toplevel 与编辑器路径比较；同名文件夹不代表同一克隆。
仓库地址错了	remote -v 核对 owner/PHM；不要只看文件夹名。
文件被忽略 / 空目录	check-ignore -v、ls-files 检查；Git 不跟踪空目录。
认证或 403	作者姓名不是登录账号；检查 GCM、PAT/SSH、仓库写权限和组织策略。
保护规则要求 PR	改走功能分支与 PR，按规则完成检查；不使用强推绕过。
网络超时 / DNS 错误	检查本地网络、GitHub 服务状态；保留本地提交，恢复网络后重试。

> **不要用危险命令猜原因**
>
> 删除 .git、reset --hard、clean -fd、push -f 会改变或丢失内容；关闭 SSL 校验和明文保存令牌也不能解决同步原理问题。

撤销与找回：先保护现场

本页是按条件选择的操作，不是连续执行的修复脚本。

依据：S14 / S15 | 完整链接见文末

只是暂存错了

```
git restore --staged -- "实际文件路径"
# 已有提交的仓库：撤回暂存，不丢工作区内容
```

已经 push 的普通提交要撤销

```
git log --oneline -10
git show 实际提交编号
git revert 实际提交编号
# 先核对目标；检查反向提交的结果，再 git push
```

revert 新建一个反向提交，保留共享历史。可能冲突，也可能影响后续依赖；合并提交的撤销需要另选主线，不能直接套用。秘密泄露仍需先撤销或轮换凭据。

已经提交过，后来找不到了

```
git reflog -15
git show 实际找到的提交编号
git branch rescue/recovered 实际找到的提交编号
```

> **reflog 是本机记录**
>
> 先核对再建救援分支，避免直接覆盖当前分支。reflog 不永久保留，也不是跨电脑日志；从未提交、没有备份的文件不能保证找回。merge / rebase 中止使用对应的 --abort。

代码能拉到，环境不一定能复制

PHM 的代码、数据与运行环境要分开管理。

依据：S11 / S18 | 完整链接见文末

纳入 Git	源码、配置示例、README、测试、按项目使用的依赖文件与锁文件。
本机重建	虚拟环境、缓存、运行日志；不把 A 的 .venv 原样搬到 B。
独立保存	原始数据、视频、权重：记录版本、来源与校验值；按授权共享。
不写进仓库	令牌、私钥、真实 .env。只放无秘密的 .env.example。
可复现实验	还需对齐数据、随机种子、依赖与 Python 版本，不只对齐代码。

关于当前 OneDrive 路径

当前学习材料位于 OneDrive 下，不代表已经出现故障。作为预防性安排，建议工作克隆使用普通本地目录，两台电脑各自 clone，经 GitHub 同步，减少云盘与 Git 同时写 .git 的风险。本次没有移动目录。

> **大文件与秘密不是普通同步错误**
>
> 常规 Git 上传超过 100 MiB 的文件会被阻止；超过 50 MiB 有警告，浏览器单文件上限 25 MiB。以官方现行限制为准，不把 LFS 当作无限免费存储。

.gitignore 不移除已跟踪内容，也不抹掉旧历史。新提交里删除大文件或秘密，仍不能清除旧提交里的副本。先备份、撤销泄露凭据，再针对具体历史制定清理方案。

每次换电脑前后，只核对这几项

把这一页打印或保存在桌面，形成固定交接习惯。

依据：S03 / S04 / S09 | 完整链接见文末

开工：确认 → 拉取 → 核对

```
git branch --show-current
git status
git pull --ff-only
git log -1 --oneline
```

收工：保存 → 暂存检查 → 提交 → 推送

```
git add -- "实际文件路径"
git diff --cached
git commit -m "描述这次修改"
git push
git status -sb
```

> **网页最后确认**
>
> 选同一分支，看到相同提交编号。交接记录：仓库、分支、最后提交短编号、push 是否成功、待做事项、检查情况、未纳入 Git 的文件。

三个问题，检验自己是否掌握

1. A 已 commit 但没 push，B 能拉到吗?
2. working tree clean 能证明上传了吗?
3. stash 中的文件会跟着普通 push 到 B 吗?

答案：都不能。交互练习见 lessons/0001-two-computers.html；详细命令与排错见同目录的 Markdown 分类教程。

官方资料与适用范围

核对日期：2026-10-19。S 编号对应各页依据；点击标题可打开官方原文。

S01 GitHub 仓库的组成

https://docs.github.com/en/repositories/creating-and-managing-repositories/about-repositories

S02 浏览 GitHub 代码与文件

https://docs.github.com/en/repositories/working-with-files/using-files/navigating-code-on-github

S03 Desktop 分支同步

https://docs.github.com/en/desktop/working-with-your-remote-repository-on-github-or-github-enterprise/syncing-your-branch-in-github-desktop

S04 git pull：快进与整合

https://git-scm.com/docs/git-pull

S05 Pro Git：Git 基础与快照

https://git-scm.com/book/en/v2/Getting-Started-What-is-Git%3F

S06 Pro Git：Git 对象

https://git-scm.com/book/en/v2/Git-Internals-Git-Objects

S07 rebase 与共享历史

https://docs.github.com/en/get-started/using-git/about-git-rebase

S08 stash：临时保存工作

https://git-scm.com/docs/git-stash

S09 GitHub flow：分支与 PR

https://docs.github.com/en/get-started/using-github/github-flow

S10 GitHub 身份认证

https://docs.github.com/en/authentication/keeping-your-account-and-data-secure/about-authentication-to-github

补充来源与制作说明

核对日期：2026-10-19。S 编号对应各页依据；点击标题可打开官方原文。

S11 大文件限制

https://docs.github.com/en/repositories/working-with-files/managing-large-files/about-large-files-on-github

S12 仓库设置

https://docs.github.com/en/repositories/managing-your-repositorys-settings-and-features/managing-repository-settings

S13 仓库活动与数据

https://docs.github.com/en/repositories/viewing-activity-and-data-for-your-repository

S14 git revert：反向提交

https://git-scm.com/docs/git-revert

S15 git reflog：本机引用记录

https://git-scm.com/docs/git-reflog

S16 GitHub 网页上传文件

https://docs.github.com/en/repositories/working-with-files/managing-files/adding-a-file-to-a-repository

S17 HTTPS 凭据缓存与 GCM

https://docs.github.com/en/get-started/git-basics/caching-your-github-credentials-in-git

S18 管理个人访问令牌

https://docs.github.com/en/authentication/keeping-your-account-and-data-secure/managing-your-personal-access-tokens

S19 SSH 密钥与 agent

https://docs.github.com/en/authentication/connecting-to-github-with-ssh/generating-a-new-ssh-key-and-adding-it-to-the-ssh-agent

S20 Desktop 推送提交

https://docs.github.com/en/desktop/making-changes-in-a-branch/pushing-changes-to-github-from-github-desktop

本手册的核实边界

已阅读两份输入材料并参考官方文档。未提供真实仓库 URL；未核实账号权限、实际分支和仓库规则。PHM 页面为教学示意。教程不等于已在用户仓库执行；本地隔离演练与版式检查见验证记录。